Movement Quest Project Report

Mohammed Afeefuddin and Muhammad Samad Khan | CODING MASTER

CSC4360 MAD | Mon-Wed | Undergraduate | October 24, 2026

A routine we can record one set at a time

We built Movement Quest around a simple task: choose a short routine, record what was actually completed, and return to those records later. A plan expresses a target. A training log records the result. Keeping those separate makes the app useful even when the actual repetitions differ from the original plan.

The interface uses a calm green theme and plain actions such as Create plan, Save plan and Finish session. Its quest theme gives the activity a friendly identity without competition, fitness prescriptions or medical claims.

Team responsibilities

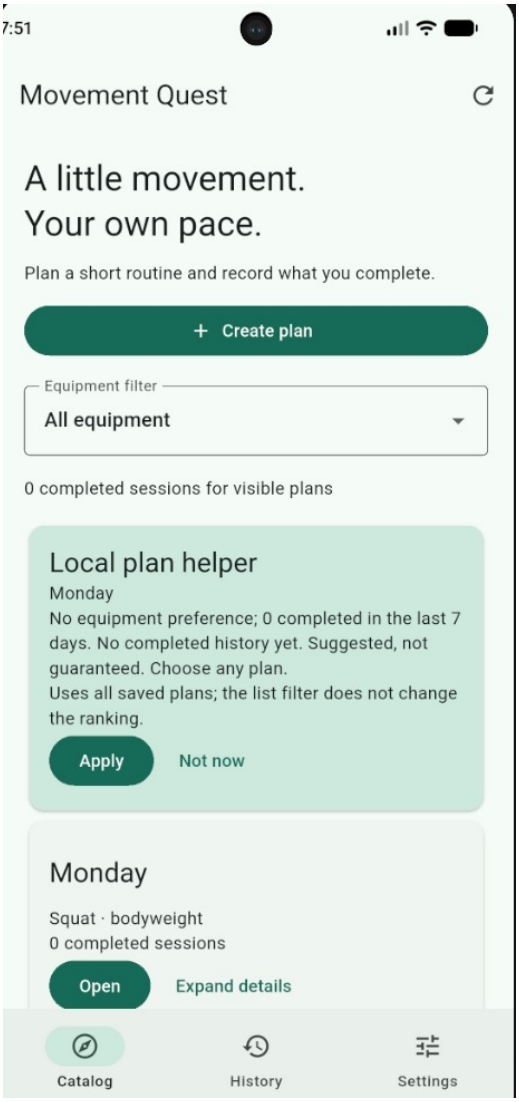
Member	Established responsibility
Mohammed Afeefuddin	Lead/UI: navigation, forms, settings, demo coordination, accessibility and integration review.
Muhammad Samad Khan	Data and quality: SQLite, CRUD, activity integrity, helper evaluation, tests, documentation and release.

These are the original ownership assignments. Actual completed contributions must be supported by each member's task, commit and review evidence. AI-assisted implementation and document preparation are disclosed in AI_Usage_Log.md.

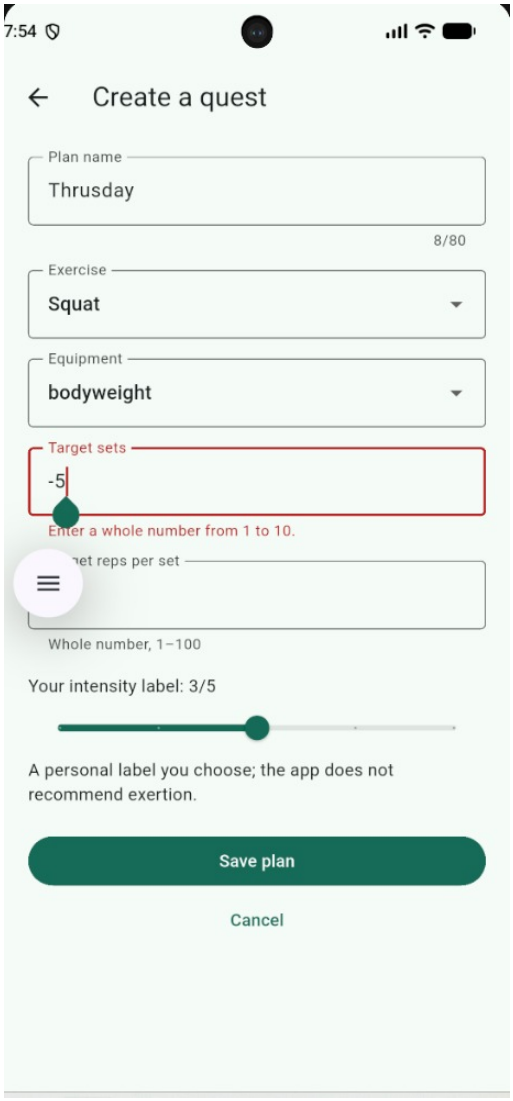
Current evidence

Afeef reports running the APK on two different devices. The October 24 captures show the app screens reproduced in this report. The earlier source helper evaluation passed 10 of 10 assertions and host SQLite schema checks passed 9 of 9. Complete Flutter test output and offline/restart results have not yet been recorded in this package.

Planning a short routine



Catalog with a saved Monday plan, equipment filter and explainable local suggestion.

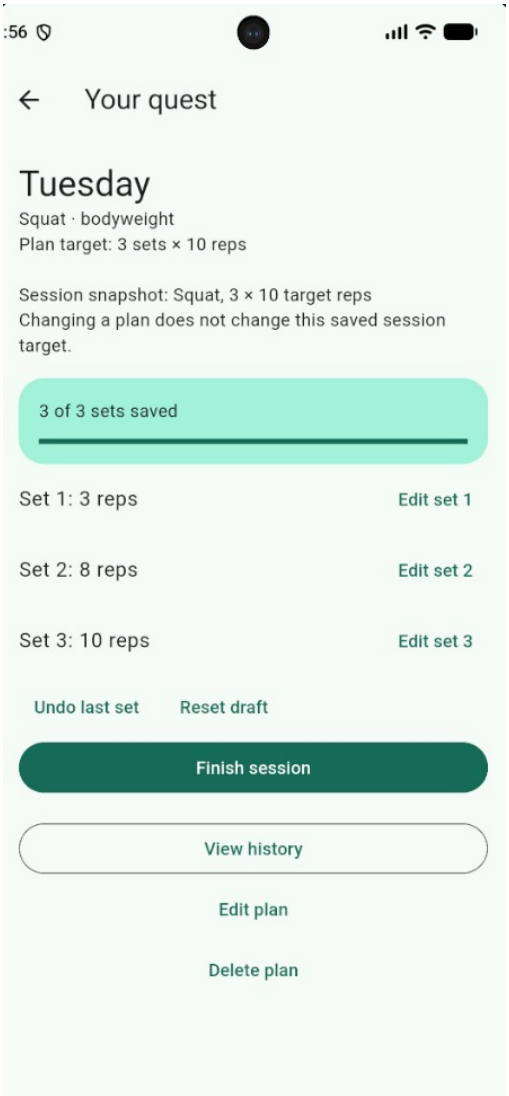


The create form rejects -5 target sets and explains the accepted range.

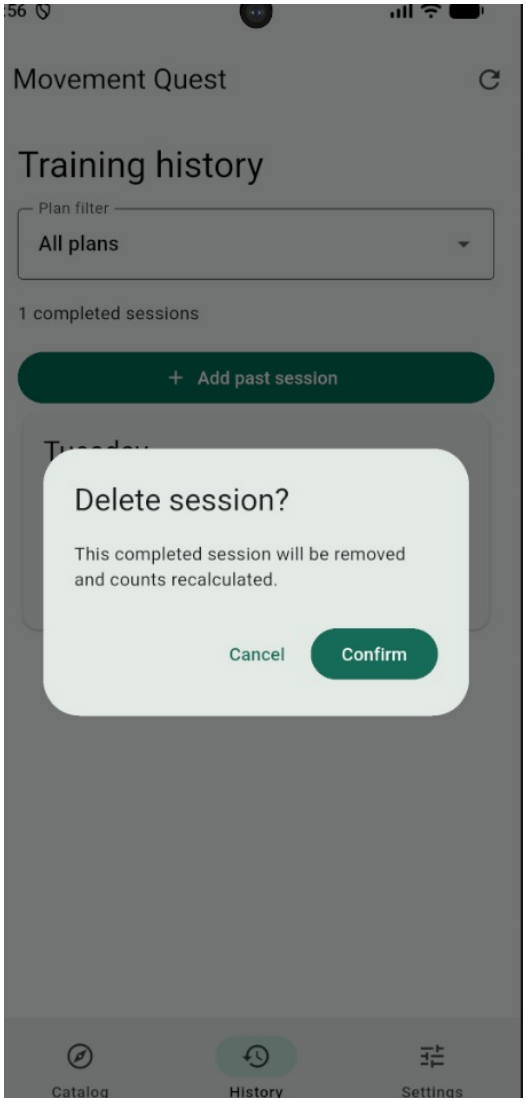
Afeef's UI responsibility

Afeef's assigned area connects navigation, form feedback and settings. The captured form puts the numeric error beside the affected input, while the catalog keeps the helper and manual plan choices available together. A final accessibility check should cover larger text, scrolling and control labels on both tested devices.

Recording actual work



Tuesday targets three sets of ten. The saved actual values are 3, 8 and 10. The screen still offers Finish session.



History shows one completed session behind a confirmation dialog. The image does not prove that deletion was confirmed.

Samad's data responsibility

Samad's assigned area protects the relationship between plans and their logs. The available source snapshots session targets, persists each saved set and prevents multiple drafts for one plan. Finishing updates the same session row. These mechanisms need native-device verification in addition to the visible progress screen.

Architecture and persistence

The five screen roles are Catalog, Plan Editor, Workout Detail, Training History and Settings. Screens manage local input. QuestController refreshes shared records after writes. QuestRepository owns SQLite, and SettingsStore owns local preferences. The helper is a pure Dart service with a controllable clock for tests.

Concern	Implementation in available source
Plans and logs	workout_plans and training_logs; foreign key, cascade deletion and plan_id index.
Session integrity	One draft per plan, stable token, transactional Start, expected-list-length checks and idempotent Finish.
Snapshots	Exercise and target values are copied into the session so subsequent plan edits do not rewrite history.
Settings	Theme, equipment preference and reduced motion stored as one validated local JSON preference.
Lifecycle	Dispose text controllers, check mounted after asynchronous work, and await repository close.
Device independence	Each installation has its own SQLite and preferences. No account, Firebase or cross-device synchronization.

Evidence boundary

The screenshots agree with the available source's main screen structure. They do not expose the database or prove that the phone APK uses the exact same revision. The latest working source, release APK and resolved dependency lockfile are needed for a final source-to-release comparison.

Design choices

A compact ChangeNotifier boundary keeps state flow explicit for this project. Actual set counts appear as text as well as progress color. Optional expansion, set and completion animations respect reduced motion in the available source. Settings, restart persistence and animation behavior still require direct acceptance checks.

An explainable local helper

Our helper organizes saved plans using equipment preference and recent completed-session counts. It is a rule-based feature, not a trained model or an exercise prescription. Score = 3 x equipment match - min(recent count, 2). The seven-day window includes its boundary; ties choose the lowest plan ID. The baseline always selects the lowest plan ID.

Measure	Recorded result
Expected-output assertions	10/10 passed in the earlier pure Dart source evaluation.
Synthetic preference agreement	Helper 6/7; baseline 4/7 on the same seven valid labeled fixtures.
Excluded from agreement	One empty-catalog and two invalid-input cases.
Case coverage	Normal, no catalog, no history, invalid preference/history, inclusive and outside boundary, tie, cap and counterexample.
Interpretation	Small hand-authored cases. No user-study, held-out accuracy or health claim.

Why user control matters

One counterexample labels a recently used plan as the user's preferred choice while the novelty penalty selects another. The implementation follows its rule correctly, but its recommendation does not capture that preference. This is why the card explains its reason and keeps Apply, Not now and manual selection available. Apply opens a plan; starting a session is a separate action.

Alternative and limitation

Nearest-neighbor matching would require example coverage and a meaningful distance definition. The current fields favor inspectable rules. A future familiar-versus-new preference could address the counterexample, but would need fresh evaluation rather than a promise of better recommendations.

Verification and remaining work

Evidence	What it establishes
Supplied images	Catalog and editor rendering, negative-set validation, saved-set display, history and delete confirmations.
User report	APK ran on two devices; models and OS versions not supplied.
Helper and host SQL runs	Source-level checks only; JSON results included.
Still unverified here	Flutter analysis/tests; full CRUD and settings checks; offline force-stop/relaunch; release/source match.
Team evidence remaining	Peer observation/revision, actual contribution traces, missing board dates, personal reflections and flow measurements.

Afeef and Samad in the demonstration

Afeef should explain the form-to-screen flow and show how clear feedback helps the user correct input. Samad should explain how writes become durable records and why a completed session cannot be duplicated. Each should connect one actual task to a commit, the relevant code and a test or review result. These are demonstration responsibilities, not invented accounts of completed work.

Release limitations

The app supports one exercise per plan and stores data locally. Devices do not synchronize. Confirmed parent deletion removes related history. The helper does not assess exercise suitability. Personal reflections and peer-review outcomes must describe what the members actually experienced.

Submission readiness

Before final submission, attach the actual approved proposal, official signed statement, tested APK and accessible recording URL. Complete Kanban evidence with genuine records, including the final October 28 snapshot. Upload selected questions before recording. Both members submit individually by October 28 at 11:59 PM Eastern.

Sources: supplied course instructions and proposal; available Movement Quest source; saved helper/schema result files; user-provided October 24 app captures and two-device run report.